Engineering Analysis: AZD vs. API Endpoints for Azure Resource Provisioning & Deployment

Azure Functions VS Code Extension Date: March 2, 2026 **Author:** Engineering Analysis — Generated Report

1. Executive Summary

This report evaluates two approaches for provisioning Azure resources and deploying code in the Azure Functions VS Code extension:

1. **API Endpoints (Current):** Direct ARM SDK calls for provisioning; Kudu SCM API for deployment
2. **Azure Developer CLI (AZD):** CLI-based provisioning via Bicep templates; CLI-based deployment via `azd deploy`

The analysis covers architecture, error handling, performance, maintainability, and user experience tradeoffs. The current prototype on branch `nat/azdResourceProvisioning` demonstrates the AZD provisioning approach with ~691 lines of new code replacing 6–8 individual ARM SDK step classes.

2. Architecture Overview

2.1 Current Architecture: API Endpoints

Provisioning (ARM SDK)

The current provisioning flow uses the `AzureWizard` framework from `@microsoft/vscode-azext-utils` to orchestrate 6–8 sequential ARM SDK calls, each implemented as an `AzureWizardExecuteStep` subclass:

Step	Priority	SDK Package	ARM Resource
ResourceGroupCreateStep	100	`@azure/arm-resources`	`Microsoft.Resources/resourceGroups`
UserAssignedIdentityCreateStep	101	`@azure/arm-msi`	`Microsoft.ManagedIdentity/userAssignedIdentities`
AppServicePlanCreateStep	120	`@azure/arm-appservice`	`Microsoft.Web/serverfarms`
StorageAccountCreateStep	130	`@azure/arm-storage`	`Microsoft.Storage/storageAccounts`
LogAnalyticsCreateStep	134	`@azure/arm-operationalinsights`	`Microsoft.OperationalInsights/workspaces`
AppInsightsCreateStep	135	`@azure/arm-appinsights`	`Microsoft.Insights/components`
RoleAssignmentExecuteStep	900	`@azure/arm-authorization`	`Microsoft.Authorization/roleAssignments`
FunctionAppCreateStep	1000	`@azure/arm-appservice`	`Microsoft.Web/sites`

These steps are spread across two repositories (`vscode-azuretools` and `vscode-azurefunctions`) and execute sequentially based on priority number.

Deployment (Kudu API)

The current deployment flow supports 6 distinct strategies selected at runtime:

Strategy	API Endpoint	When Used
Kudu ZIP Deploy	`POST /api/zipdeploy`	Default for most apps
Kudu WAR Deploy	`POST /api/wardeploy`	Tomcat/WildFly/JBoss runtimes
Flex Consumption Publish	`POST /api/publish`	Flex Consumption SKU
Storage Account Deploy	Azure Blob Storage upload	Linux Consumption without remote build
Local Git Deploy	`git push` to SCM endpoint	User-configured LocalGit SCM type
func CLI Publish	`func azure functionapp publish`	Custom runtimes on Flex

Deployment tracking is done by polling `GET /api/deployments/{id}` with structured log retrieval from `GET /api/deployments/{id}/log`.

2.2 Proposed Architecture: AZD

Provisioning

A single `AzdProvisionExecuteStep` replaces all individual ARM steps:

1. Generates a Bicep template inline via `generateBicepTemplate()` with all resources defined (no modules)
2. Generates an `azure.yaml` manifest via `generateAzdYaml()`
3. Writes both to a temporary directory with AZD environment configuration
4. Pre-creates the resource group via ARM SDK
5. Runs `azd provision --no-prompt` via `AzdProvisioningRunner`
6. Retrieves the created Function App from ARM to populate the wizard context

Output is streamed to a custom tree view (`AzdProvisioningTreeDataProvider`) that shows per-resource status with spinner icons.

Deployment

No AZD deployment integration exists in the current prototype. A hypothetical `azd deploy` integration would replace the Kudu API calls with a single CLI invocation.

---

3. Provisioning Analysis

3.1 Benefits of AZD Provisioning

3.1.1 Atomic Deployment with Transactional Semantics

ARM template deployments are submitted as a single deployment operation. If any resource fails, ARM can roll back the entire deployment. The current sequential SDK approach can leave partial resources (e.g., a storage account created but the Function App creation fails) requiring manual cleanup.

3.1.2 Declarative Infrastructure

The `generateBicepTemplate()` function produces a single Bicep file with all resources inline. Adding a new resource means adding Bicep syntax rather than writing a new `AzureWizardExecuteStep` subclass, wiring dependencies, and managing priority ordering. This is substantially less code per resource.

3.1.3 Automatic Dependency Ordering

The current approach requires manually assigning priority numbers (100, 120, 130, 134, 135, 900, 1000) and reasoning about sequential execution order. Bicep's implicit dependency graph (inferred from resource references) eliminates this concern and

enables ARM to parallelize independent resources — for example, the Storage Account and Log Analytics Workspace can be created concurrently.

3.1.4 Enhanced Progress Visibility

The `AzdProvisioningTreeDataProvider` and `AzdProvisioningRunner` provide real-time per-resource status tracking parsed from `azd` output, with spinner icons, success/failure markers, and session history — a richer experience than the current generic activity log entries.

3.1.5 Reduced Code Duplication

The ARM steps are spread across 2 repositories. Changes to provisioning logic currently require coordinated releases of `vscode-azuretools` and `vscode-azurefunctions`. The AZD path consolidates provisioning into the extension's own codebase.

3.1.6 Ecosystem Alignment

AZD is Microsoft's strategic investment for developer provisioning workflows. Adopting it aligns with the broader Azure DevEx direction and means the extension benefits from AZD improvements without code changes.

3.2 Drawbacks of AZD Provisioning

3.2.1 External CLI Dependency

The current approach has zero external dependencies — all ARM SDK packages are bundled. The AZD path requires `azd` to be installed on the user's machine. While there is a graceful fallback to the ARM path, this introduces:

- **Feature fragmentation:** Users without `azd` get a different experience
- **Version skew risk:** Generated Bicep syntax may not be compatible with all `azd` versions
- **No control over CLI bugs or breaking changes**

3.2.2 Loss of Granular Error Handling

The ARM steps have sophisticated per-resource error handling:

Step	Error Handling
ResourceGroupCreateStep	Catches 403 → falls back to existing RG selection
AppServicePlanCreateStep	Catches AuthorizationFailed → prompts to select existing
AppInsightsCreateStep	Checks location compatibility, skips if unsupported
RoleAssignmentExecuteStep	5 retries for AAD replication delay

The AZD path has a single failure point. If `azd provision` fails, the user sees a terminal error with an exit code — no per-resource recovery, no interactive fallback, no retry logic for known transient issues.

3.2.3 Debugging Difficulty

When an ARM SDK call fails, the extension inspects HTTP status codes, parses error bodies, and presents meaningful messages. With AZD, errors come as terminal output parsed via regex (`AZD_PROVISION_FAILURE_RE`). If `azd` 's output format changes, the tree view parsing breaks silently.

3.2.4 Shell Integration Fragility

The `AzdProvisioningRunner` depends on VS Code's `TerminalShellIntegration` API with a 10-second activation timeout and fallback to `sendText()` without output tracking. This may not work reliably in all environments (SSH remotes, certain terminal types).

3.2.5 Temporary File Management

The step creates a temporary AZD project structure on disk, writing `azure.yaml`, Bicep files, and `.azure/` environment config. Best-effort cleanup is attempted, but VS Code crashes or antivirus interference could leave artifacts.

3.2.6 No Advanced Creation Support

The AZD path only covers basic (non-Docker) creation. Advanced creation still uses the ARM path, meaning both code paths must be maintained indefinitely — increasing test surface area.

3.2.7 Custom Cloud Concerns

Hardcoded Bicep API versions (e.g., `Microsoft.Web/sites@2023-12-01`) may not be available in all sovereign or custom cloud environments. The ARM SDK approach uses per-cloud API version negotiation.

3.2.8 Overhead for Simple Cases

The AZD approach adds overhead: write temp files → spawn CLI → parse Bicep → compile to ARM template → submit deployment → poll. For a simple Function App, direct SDK calls are likely faster.

3.3 Provisioning Summary

Dimension	ARM SDK (Current)	AZD (Proposed)
ARM calls	6–8 sequential SDK calls	1 RG create + 1 CLI invocation + 1 site GET
Error handling	Per-step: 403 fallbacks, existence checks, retry	Single failure point; relies on azd error reporting
Retry logic	RoleAssignment: 5 retries for AAD replication	None (azd handles internally, opaque)
External dependency	None (bundled SDK packages)	Requires `azd` CLI on PATH
Parallelism	Sequential by priority number	ARM parallelizes independent Bicep resources
Code maintenance	~8 classes across 2 repos	3 files in 1 repo
Custom clouds	Dynamic API version negotiation	Hardcoded Bicep API versions
Rollback	Manual cleanup of partial failures	ARM template-level rollback

---

4. Deployment Analysis

4.1 Benefits of AZD Deployment

4.1.1 Unified Workflow

If provisioning already uses AZD, deployment via `azd deploy` completes the `azd up` story — provision and deploy with one toolchain. Users get a consistent mental model.

4.1.2 Language-Aware Build and Packaging

`azd deploy` understands project types and runs appropriate build and package steps automatically. The current extension has a complex chain of conditionals to determine the deploy method based on runtime, OS, SKU, and configuration settings.

4.1.3 Simpler Code Surface

The current deploy path spans approximately 15 files across two repositories with 6 different mechanisms. An AZD path would be a single execute step.

4.1.4 Native Remote Build Handling

`azd deploy` handles Oryx remote builds without the extension needing to manage `SCM_DO_BUILD_DURING_DEPLOYMENT` or `WEBSITE_RUN_FROM_PACKAGE` app settings.

4.2 Drawbacks of AZD Deployment

4.2.1 Incomplete Strategy Coverage

The current system supports 6 deployment strategies. AZD does not cover all of them:

Strategy	AZD Support
Kudu ZIP deploy	Likely supported
Kudu WAR deploy	Uncertain — may not support WAR deploy to App Service
Flex Consumption publish	Version-dependent
Storage Account deploy	No equivalent — this sets `WEBSITE_RUN_FROM_PACKAGE` to a blob SAS URL
Local Git deploy	Not applicable — fundamentally different mechanism
func CLI publish	AZD may or may not call `func` under the hood

4.2.2 Loss of Retry and Polling Infrastructure

The current Kudu flow has purpose-built retry and polling logic:

- `retryKuduCall()` — 4 retries with exponential backoff on all deployment status polling
- `waitForDeploymentToComplete()` — structured polling at 1-second intervals with 60-second initial timeout
- `syncTriggersPostDeploy()` — 5 retries with 5-second minimum backoff
- `ignore404Error()` — silently swallows transient 404s during log fetching

With AZD, the extension only sees the process exit code and stdout.

4.2.3 Loss of Deployment Log Streaming

The current flow fetches individual log entries from `/api/deployments/{id}/log`, cleans Oryx duplicate lines, and reports them in VS Code's output channel with clear formatting. AZD logs would be raw terminal output requiring regex parsing.

4.2.4 Loss of VS Code Integration Hooks

The current deploy wizard integrates deeply with VS Code:

- **Pre-deploy task** — runs the user's configured build task
- **Post-deploy task** — runs cleanup/notification tasks
- **App settings verification** — checks runtime version parity before deploying
- **Trigger sync** — calls ARM `syncFunctionTriggers()` after deploy
- **HTTP URL listing** — queries and displays function trigger URLs

These hooks would need to be maintained externally, partially offsetting the simplicity benefit.

4.2.5 Critical Path External Dependency

Deploying code is the highest-frequency user action. Making it depend on an external binary is riskier than for provisioning. If `azd` has a bug, users cannot deploy until it is fixed — and the extension team has no control over the fix timeline.

4.2.6 Authentication Model Mismatch

Kudu calls use the Azure identity token from the user's VS Code sign-in session. `azd` manages its own authentication via `azd auth login`. If the two sessions diverge, deployments could target the wrong subscription or fail with confusing auth errors.

4.2.7 Performance Overhead

Direct Kudu ZIP deploy is a single HTTP POST. `azd deploy` spawns a process, reads `azure.yaml`, resolves the service, zips the project, and then calls the underlying endpoint. For rapid inner-loop workflows, this overhead accumulates.

4.2.8 `.funcignore` Support Uncertainty

The current `runWithZipStream()` respects `.funcignore` files. AZD uses its own ignore logic. Behavioral differences would cause inconsistent deployment contents depending on which path runs.

4.3 Deployment Summary

Dimension	Kudu API (Current)	AZD Deploy (Hypothetical)
Deploy strategies	6 strategies for all app types	Primarily zip-based; gaps for WAR, Storage, LocalGit
Retry/resilience	4-retry polling + trigger sync retries	Opaque — depends on azd internals
Log streaming	Structured per-entry log fetching	Raw terminal output parsing
VS Code integration	Deep (tasks, settings, output, notifications)	Shallow (process stdout only)
External dependency	None (all in-process)	Requires `azd` CLI installed
Authentication	In-process Azure identity token	Separate `azd auth` session
Maintenance cost	High (~15 files across 2 repos)	Low (single step) + wrapper hooks
Inner-loop speed	Fast (direct HTTP POST)	Slower (process spawn + packaging overhead)

---

5. Recommendations

5.1 Provisioning: AZD is a promising direction

The AZD provisioning approach offers meaningful benefits in atomicity, parallelism, and maintainability. It is worth pursuing with these mitigations:

1. **Structured output** — Use `azd provision --output json` (if available) instead of regex parsing terminal text
2. **AAD replication retry** — Add post-provision retry logic for role assignment propagation or use a Bicep `dependsOn` delay mechanism
3. **Version pinning** — Check minimum `azd` version in `isAzdInstalled()`, not just CLI presence
4. **Feature parity gating** — Until advanced creation and Docker paths are covered, clearly document that the ARM path remains canonical

5.2 Deployment: Kudu API should remain primary

For deployment, the Kudu API path has significantly stronger advantages. The architecture is battle-tested with per-strategy optimizations, robust retry logic, deep VS Code integration, and zero external dependencies. AZD deploy's complexity reduction is partially negated by the need to maintain pre/post-deploy hooks, trigger syncing, and app settings verification externally.

Recommendation: Offer AZD deploy as an opt-in alternative for users already using `azd` end-to-end, rather than replacing the Kudu path.

5.3 Hybrid Strategy

The strongest path forward is a **hybrid approach**:

Operation	Recommended Approach	Rationale
Provisioning (basic)	AZD (with ARM fallback)	Atomic deployments, parallelism, less code
Provisioning (advanced/Docker)	ARM SDK	AZD doesn't cover these paths yet
Deployment	Kudu API (primary)	Mature, reliable, deeply integrated
Deployment (AZD users)	AZD deploy (opt-in)	Consistent `azd up` experience for power users

---

6. Risk Matrix

Risk	Severity	Likelihood	Mitigation
`azd` CLI not installed	Medium	High	Graceful fallback to ARM/Kudu (already implemented)
`azd` output format changes	High	Medium	Use structured JSON output; add integration tests
AAD replication delay breaks role assignment	Medium	Medium	Add post-provision retry; Bicep delay mechanism
Shell integration fails in remote environments	Medium	Low	Fallback to `sendText()` (already implemented)
Sovereign cloud API version mismatch	High	Low	Per-cloud Bicep parameter overrides
Temp file cleanup failure	Low	Low	OS-level temp directory cleanup; add TTL-based cleanup
Auth session divergence (deploy)	High	Medium	Share auth context or validate before deploy

---

End of Report